OPSFLEET

Retail Data Analysis Assistant

High Level Design

Requirements, component relationships, architectural decisions and conclusions

Production architecture and POC scope

Prepared for OpsFleet

November 2026

OpsFleet Retail Data Analysis Assistant

Requirements, Component Relationships, Decisions and Conclusions

Document status: Proposed production High-Level Design (HLD), with Proof of Concept (POC) delivery obligations identified.
Source material: OpsFleet assignment, requirements matrix screenshot, production logical and GCP deployment diagrams, runtime dataflow diagram.
Framework: LangGraph (Python); **analytics:** BigQuery; **LLM:** Gemini; **target cloud:** Google Cloud Platform.
Implementation status caveat: The POC obligations below are established by the assignment. This document does **not** certify that the current repository implements them. A separate code review, tests and demo run are needed to establish actual completion.

---

1. Executive overview and scope

The proposed assistant lets retail executives ask questions about sales, products, customers and business performance using natural language. It translates authorized analysis requests into BigQuery SQL, validates queries before execution, interprets returned evidence, carries conversational context across follow-up questions, and generates executive reports with conclusions and recommended actions. It also manages a saved-report library; deleting reports is allowed only within a user's ownership scope and requires explicit confirmation.

The production architecture combines a deterministic security and data-access layer with LLM-driven planning and synthesis. LangGraph coordinates the workflow and durable human-in-the-loop pauses. Historical expert examples (Golden Trios), user preferences and evaluation feedback provide controlled improvement over time. The GCP deployment design places stateless application compute on Cloud Run; transactional application state in Cloud SQL for PostgreSQL; analytical data in BigQuery; Golden Knowledge documents and exports in Cloud Storage; and model inference, embeddings and retrieval infrastructure in Vertex AI. These are **design choices**, not proof of an existing deployment.

1.1 Requirement coverage matrix

The matrix below follows the supplied screenshot. "POC status" indicates the **required demonstration commitment**, not an independently verified implementation result.

R01

Requirement: Natural-language SQL analysis

POC commitment: Required: implement and demo

Production HLD: Required

R02

Requirement: Conversational follow-up questions

POC commitment: Required: implement and demo

Production HLD: Required

R03

Requirement: Analytical reports and action items

POC commitment: Required: implement and demo

Production HLD: Required

R04

Requirement: Safety, PII and access control

POC commitment: Required: implement and demo

Production HLD: Required

R05

Requirement: Destructive operations with confirmation

POC commitment: Required: implement and demo

Production HLD: Required

R06

Requirement: Error recovery and resilience

POC commitment: Required: implement and demo

Production HLD: Required

R07

Requirement: Observability

POC commitment: Required: implement and demo

Production HLD: Required

R08

Requirement: Golden Knowledge retrieval

POC commitment: Optional in POC

Production HLD: Required

R09

Requirement: User preference learning

POC commitment: Optional in POC

Production HLD: Required

R10

Requirement: System learning loop

POC commitment: Optional in POC

Production HLD: Required

R11

Requirement: Automated quality evaluation

POC commitment: Optional in POC

Production HLD: Required

R12

Requirement: Dynamic persona management

POC commitment: Optional in POC

Production HLD: Required

R13

Requirement: Email, charts and external tools

POC commitment: Extensibility only in POC

Production HLD: Describe architecture

The assignment explicitly requires a **working CLI**, reproducible setup on another machine, and BigQuery integration with the four tables orders, order_items, products, and users. These are cross-cutting deliverable constraints rather than additional rows in the screenshot.

1.2 POC status interpretation

The only verified development baseline reported by the project owner is an existing uv Python project, an assignment-provided BigQuery Runner class, a basic tested Gemini chat function, a .env file and a Git repository. Gemini 3.8 Flash experienced timeouts under load. These facts do not establish completion of R01–R07 end-to-end. Therefore each mandatory requirement below is marked **"Required in POC: yes; actual implementation/demo: not independently verified."** Optional requirements are **"Required in POC: no; proposed production design."**

---

2. Individual requirements

R01 — Natural-language SQL analysis

What it consists of. Translating an executive's natural-language question into a meaningful analytical plan, one or more syntactically valid GoogleSQL queries, authorized BigQuery execution, and a comprehensible answer grounded in the returned data. It includes schema-aware joins, metric definitions, filters, time windows, aggregates, comparisons and multi-step queries. It also covers questions about what the database contains and what analyses it can support.

Purpose. Remove SQL as a prerequisite for executive self-service analytics while retaining the accuracy, accountability and repeatability expected from an analytical system.

Design and provision. The API receives a verified user request. The LangGraph intent router selects the analysis subgraph. Schema discovery and the business semantic layer provide approved table relationships and metric rules; Golden Knowledge retrieval may provide analogous historical analyses. A planner decomposes a complex request into subquestions. Gemini produces structured SQL proposals. A deterministic SQL/access validator parses and validates each proposal; the BigQuery Runner executes only approved queries with bounded cost, time and result size. The result-analysis node compares returned data with the question and passes supported observations to the report/synthesis node. The final output includes the time period, metric definition, notable findings and limitations. A schema-information route can respond without executing arbitrary user-provided SQL.

Decisions and assumptions. Use a single LangGraph orchestration workflow with specialist nodes rather than a collection of independent autonomous agents. Use order_items.sale_price as a candidate transaction-price field, but **the actual revenue metric and return/cancellation treatment must be agreed**. The assistant must not claim causality from simple observational decompositions. A semantic layer is recommended even in a small implementation. The LLM proposes SQL; it never directly executes privileged database operations. BigQuery is read-only and only the specified tables or restricted analytical views are in scope.

POC implementation/demo. Required; not independently verified. Demonstrate time-series sales, product comparisons, state comparisons, database-schema questions and at least one multi-step analysis. Verify the numerical result independently using reference SQL.

R02 — Conversational follow-up questions

What it consists of. Maintaining the meaning of previous questions, entities, periods, metrics and reports so the executive can ask elliptical follow-ups such as “What about Texas?”, “Break that down by category”, or “Save this as a report”.

Purpose. Support natural executive dialogue and reduce repetitive restatement, without confusing the current user request with old context.

Design and provision. LangGraph state is identified by a stable thread_id and persisted through a checkpointer. The conversation state holds a bounded message history, current intent, prior

analytical definitions, safe summary statistics, prior report references and pending human approvals. A follow-up resolver turns contextual references into an explicit analytical request before planning. Long histories are summarized selectively; expensive raw query results should be stored separately or summarized, not copied in full into every checkpoint. On a new request, the access scope is revalidated rather than blindly inherited from earlier turns.

Decisions and assumptions. In production, use PostgreSQL-backed checkpoints. In the POC, a local durable store is acceptable if resume behavior can be demonstrated. A user preference store is separate from conversation checkpoints (see R09). Thread identifiers must be scoped to authenticated ownership. Conversation memory is not authorization evidence.

POC implementation/demo. Required; not independently verified. Ask an initial question, follow with a pronoun/time-window variation, and confirm the generated SQL and answer preserve the correct context while enforcing the same product permissions.

R03 — Analytical reports and action items

What it consists of. Producing reusable reports from analytical evidence, including an executive summary, methods/metric definitions, numeric findings, comparisons, uncertainties, business implications and actionable recommendations. Reports can be viewed, saved, listed and retrieved.

Purpose. Turn isolated answers into decision-ready outputs that executives can revisit and discuss.

Design and provision. The synthesis node consumes approved query results and constructs a structured report object. A report manager validates ownership and persists metadata and content in the saved-report store (Cloud SQL in production; local database in the POC). Optional PDF/CSV exports can be stored in GCS. Each report carries a stable ID, owner/tenant, originating thread, creation date, source query/job references, metric definitions, persona version and evidence provenance. Recommendations are separated from proven observations; unsupported claims are rejected or qualified.

Decisions and assumptions. The system generates executive-level interpretation, not guaranteed causal proof. The report store is writable even though analytical BigQuery access is read-only. Reports should be versioned or immutable once issued, with revisions stored explicitly. For the POC, a well-structured CLI-rendered report plus persistence is enough; exporting PDF is not mandatory.

POC implementation/demo. Required; not independently verified. Generate a Q1/Q2-style report with several data-backed findings and concrete actions, save it, list it and retrieve it.

R04 — Safety, PII protection and product-level access control

What it consists of. Restricting the assistant to authorized analytical/report tasks, protecting against malicious prompts or tool requests, forbidding direct PII disclosure, and ensuring each executive analyzes only products assigned to them.

Purpose. Preserve confidentiality, prevent misuse of the assistant, and enforce organizational segregation of data even when an LLM or user supplies unsafe instructions.

Design and provision. Defense in depth comprises (1) trusted authentication and authorization context, (2) explicit product entitlement mapping, (3) restricted analytical schema/views and read-only BigQuery permissions, (4) SQL AST and query-policy validation before execution, (5) filtering/suppression of sensitive results before LLM exposure, and (6) final-output validation. Direct identifiers from users (for example name, email, address) are excluded from permissible outputs. Privacy controls should also suppress very small aggregates and discourage iterative differencing attacks that reveal individual behavior. Golden Knowledge and database cells are treated as untrusted retrieved data, not instructions. Reports and traces receive their own privacy controls.

Decisions and assumptions. The public theLook dataset does not establish which products belong to which executive; therefore a **separate entitlement dataset** is necessary. Product ID–level entitlements are preferred, with category-level demonstration possible if clearly declared. A shared service account is not equivalent to per-executive row security; application enforcement must be backed by a trusted data-access boundary and carefully designed BigQuery views/policies. Reject unauthorized requests rather than having the model merely promise not to comply. Customer-behavior questions must be answered in an aggregation- and privacy-conscious way, despite the assignment's "top customers" example.

POC implementation/demo. Required; not independently verified. Show (a) a prohibited request for emails or addresses is rejected, (b) simulated User A cannot analyze User B's products, (c) malicious instruction content is not executed, and (d) privacy-safe aggregate questions still work.

R05 — Destructive operations with explicit confirmation

What it consists of. Recognizing deletion requests for saved reports (such as all reports mentioning a client or all reports from a conversation), identifying exact targets, and requiring a user confirmation before any destructive write.

Purpose. Prevent accidental, ambiguous or malicious loss of reports while maintaining a natural conversational experience.

Design and provision. A report-management node resolves a target set using parameterized, ownership-scoped database queries. It presents count and names (or a clear preview for large sets), stores the frozen report IDs and operation ID, and pauses the graph with a LangGraph human-in-the-loop interrupt. On resume, a trusted executor checks that the response is bound to the correct pending operation, rechecks ownership and target validity, applies an idempotent soft deletion within a transaction, and creates an audit record. Cancellation and expiration make no modifications. Associated exported artifacts must follow a defined retention/deletion policy. Generic model interpretations of "yes" are not sufficient proof of consent.

Decisions and assumptions. Soft deletion is the recommended initial design for recoverability. Confirmation is required for every destructive operation; the user may delete only their own reports. The pending operation cannot silently expand its target set after confirmation. LangGraph

nodes can re-execute on resume, so execution must be idempotent and separated from preparatory side effects.

POC implementation/demo. Required; not independently verified. Create several reports, request bulk deletion, cancel and verify preservation, then explicitly confirm and verify only owned target reports were affected. Show the audit trail.

R06 — Error recovery, resilience and graceful degradation

What it consists of. Detecting invalid SQL, empty returns, schema mismatches, service timeouts, rate limits and dependency outages, recovering when safe, and returning useful feedback without crashing the CLI or creating uncontrolled cost.

Purpose. Provide dependable executive UX and predictable operating cost despite probabilistic model behavior and distributed cloud dependencies.

Design and provision. Use typed errors and a bounded state-machine correction loop. A syntax/column error returns sanitized diagnostics to a SQL-repair node, whose new SQL must pass the entire validator again. Empty results are classified as legitimate absence, overly narrow filters, or suspected analytical mismatch; they must **not automatically** be broadened simply to obtain data. Transient HTTP or BigQuery failures use capped exponential backoff and jitter; permanent authorization or validation failures do not retry. Each request has limits on LLM calls, SQL queries, retries, tokens, BigQuery bytes billed, and wall-clock time. Use configurable timeouts, circuit-breaker behavior where appropriate, and explicit terminal messages with partial findings and next steps. Preserve conversation state across recoverable interruptions.

Decisions and assumptions. Suggested initial maximum: two SQL repair attempts; the final values should be measured. Gemini model selection remains configurable because the user observed 3.8 Flash timeouts. Do not automatically fail open to an unauthorized model endpoint or data source. A model fallback is valid only when it meets the same data-handling and policy constraints.

POC implementation/demo. Required; not independently verified. Inject a controlled invalid-column query, an empty-result scenario and a simulated model timeout; demonstrate bounded recovery or graceful failure and continuity of the CLI.

R07 — Observability and deep debugging

What it consists of. Collecting traces, logs, metrics and audit events across intent routing, retrieval, model calls, SQL validation, BigQuery execution, synthesis and report management.

Purpose. Identify when requests fail, where failures originate, how much each run costs, and whether security/quality targets are being met.

Design and provision. Assign each turn a correlation/trace ID and record spans for graph nodes and external calls. Capture node duration, model identity/version, prompt version, tokens, query validation results, BigQuery job IDs/bytes billed, row counts, retries, classification outcomes, report approvals and final status. Aggregate first-pass SQL validity, end-to-end success, grounded answer rate, PII block events, latency percentiles, token/BigQuery cost and approval cancellation rate.

LangSmith can display agent traces; Cloud Logging and Monitoring capture application/infrastructure signals. Correlate these layers with stable request IDs. Alert on elevated failure, cost and unauthorized-access patterns.

Decisions and assumptions. Logging **must be privacy-safe**: do not store raw PII, secrets, unrestricted prompts, raw customer query rows or report contents in traces by default. Audit records may have stronger retention and restricted access than ordinary debug spans. Model evaluation scores are distinct from operational telemetry.

POC implementation/demo. Required; not independently verified. Display a trace or structured log for a successful analytical turn and a failed/repaired query; show linked IDs, timing, query status, retry count and error reason without sensitive data.

R08 — Golden Knowledge retrieval (hybrid intelligence)

What it consists of. Retrieving historical human analyst **Question → SQL → Analyst Report** Trios and using them as guidance about analytical methods, query patterns, business definitions and report interpretation.

Purpose. Incorporate expert organizational knowledge rather than relying only on the LLM's generic knowledge and current BigQuery rows.

Design and provision. Store authoritative versioned Trios in GCS with metadata (business domain, schema version, tables, metrics, dates, owner, access classification, review status). An ingestion Cloud Run Job validates, redacts, embeds and indexes approved Trios using Vertex AI embeddings and Vector Search or a smaller alternative. At query time, the Golden Retriever filters by the user's authorization and relevant domain, retrieves semantically similar questions, reranks for usefulness and recency, and returns a limited selection to the planner. The model adapts the analytical *pattern*, not the historical answer or SQL verbatim. New candidate Trios come from reviewed and validated analysis; human approval prevents error amplification.

Decisions and assumptions. The assignment calls the Golden Bucket *theoretical*, so synthetic reviewed examples are a reasonable optional POC demonstration pending client clarification. The original source objects and the vector index are separate storage roles. Embeddings must not bypass document-level entitlements; examples with obsolete schema/metric definitions are filtered or marked stale.

POC implementation/demo. Not required by the screenshot; production design required. Optional demonstration: retrieve three curated example Trios for a comparable business question and show how they influence query planning. Do not claim an operational historical corpus exists unless supplied.

R09 — User preference learning

What it consists of. Persisting user-specific choices for presentation format, analytical depth, visualizations, report length, and similar non-security preferences across conversations.

Purpose. Make answers efficient and individually useful to executives without requiring them to restate formatting instructions each time.

Design and provision. Maintain a structured user-preference profile in Cloud SQL keyed by verified user and tenant. Explicit preferences are written with clear provenance and timestamp; inferred preferences require sufficient evidence, confidence thresholds, and easy correction. The orchestrator loads them after authentication and combines them with the approved persona configuration to control output formatting. Preference updates can be periodic rather than applied after every utterance.

Decisions and assumptions. Explicit preferences outrank tentative inferences. Preferences do not alter the business semantic layer, SQL authorization, privacy restrictions or allowed tool set. Users should be able to inspect, correct and reset their preferences; retention and consent requirements need governance.

POC implementation/demo. Not required; production design required. Optional example: User A consistently receives a table and User B bullet points, both backed by the same authorized data.

R10 — System learning loop

What it consists of. Using past interactions, failures, approved reports and evaluator feedback to improve retrieval, prompts, metric definitions, query planning and agent policies over time.

Purpose. Increase accuracy and reliability while preventing the accumulation of misleading or insecure learned behavior.

Design and provision. Ingest privacy-safe run outcomes into an offline improvement pipeline. Cluster recurring SQL failures, incorrect metric use, irrelevant retrievals and ungrounded report statements. Curate difficult examples and analyst corrections into a benchmark. Propose changes to prompts, semantic rules or the Golden corpus; evaluate each candidate against regression and security suites; require approval before production rollout. Capture lineage (prompt/index/model/schema versions) to attribute improvements or regressions.

Decisions and assumptions. Do not automatically promote every agent-generated answer to the Golden Bucket. No self-modification of authorization code or guardrails. Fine-tuning is a potential later option, not a default requirement. Production updates should be reversible and measured against a fixed baseline.

POC implementation/demo. Not required; production design required. A documented learning-loop pipeline and improvement criteria are sufficient.

R11 — Automated quality evaluation

What it consists of. Repeatable tests of SQL correctness, intent fulfillment, report factual grounding, privacy and security, reliability, and executive user experience.

Purpose. Establish evidence that releases are safe and that generated answers solve the question asked, not merely produce valid SQL or plausible prose.

Design and provision. Assemble a representative versioned test corpus with user questions, metric interpretations, reference SQL/results, access scopes, adverse inputs, and validated report rubrics. Evaluate SQL through execution-result equivalence and query-policy compliance rather than string matching. Grade reports for numerical correctness, completeness, faithfulness to results, explanation quality, caveats and action-item relevance; use deterministic numerical tests where possible and LLM/human judges only for qualitative dimensions. Include end-to-end fault injection and human usability tasks, e.g., multi-turn comparison and deletion cancellation. Put critical regressions in CI and run broader offline evaluations prior to release.

Decisions and assumptions. Require clear approval thresholds for security tests and a representative accuracy benchmark. A generated report can be fluent yet false; LLM-as-judge alone is insufficient. The POC should still contain basic tests for implemented security and reliability, even though the full automated QA platform is not mandatory.

POC implementation/demo. Full automated quality evaluation optional in POC; production design required. The code deliverable should still be accompanied by a focused manual/automated verification checklist for mandatory behaviors.

R12 — Dynamic persona management

What it consists of. Giving authorized nontechnical administrators a way to edit reporting tone and style at runtime, with version history and rollback.

Purpose. Meet weekly executive communication changes without redeploying or changing the core analytical/safety behavior.

Design and provision. Store approved, versioned persona templates in Cloud SQL (or a governed configuration service). An admin workflow validates and publishes changes; the agent loads an active version at run start, with an optional cache and invalidation strategy. The effective version is stamped into reports and traces. The persona governs tone, verbosity, report structure and preferred emphasis but cannot override hard safety instructions, tool policies or metric definitions.

Decisions and assumptions. Separate immutable engineering/security instructions from admin-editable style prompts. Scope persona settings globally, per tenant or per business role as appropriate. Require RBAC, audit, test preview and rollback. Do not let a user edit persona configuration simply by chatting with the analysis agent.

POC implementation/demo. Not required; production design required. An HLD description and sample configuration lifecycle suffice.

R13 — Extensibility: email, charts and external tools

What it consists of. Adding future capabilities such as visual charts, PDF exports, emailed reports, scheduled analyses and sourced external trend research without redesigning the core graph.

Purpose. Preserve architectural agility and support evolving executive workflows.

Design and provision. Define typed tool interfaces and permission models. A chart tool receives only approved aggregate data and renders a visualization. An email tool requires verified recipients, report authorization, explicit send confirmation when appropriate, and auditing. A web research tool returns attributed external information treated as untrusted; it must be kept distinct from BigQuery-derived facts. New nodes or subgraphs register with the intent router and share identity, observability, resilience and output-safety controls. Asynchronous tasks can run through Cloud Run Jobs and a suitable queue/scheduler in production.

Decisions and assumptions. Do not provide unbounded code execution, unrestricted web browsing or generic outbound messaging solely at the LLM's discretion. Explicit capabilities and policies are preferable to a general-purpose omnipotent tool. Separate analysis evidence from external trend context in final reports.

POC implementation/demo. Extensibility only; no working email/chart/web tools required. Document tool contracts and architecture extension points.

---

3. Relationships, interfaces and communication design

3.1 Relationship categories

The logical diagram shows **control flow** (LangGraph chooses a node), **data flow** (query results or retrieved context move between nodes), **persistence** (checkpoints/reports are written or read), and **external service interaction** (HTTPS or database protocol). Do not mistake every arrow for a separate network hop: many graph nodes execute as ordinary Python calls within one Cloud Run container.

The deployment diagram shows *where capabilities run*; the runtime-dataflow diagram shows *in what order a request proceeds*. Both must be read together. The relationships below identify the protocol, purpose, validation and connection management for the principal interactions.

3.2 Communication and dependency matrix

CLI/Web → API gateway

Relationship and purpose: Submit question, receive streaming or final answer, handle approval

Mechanism/protocol: HTTPS/JSON; optional SSE or WebSocket for progress

Validation and connection handling: TLS; authenticated token/session; JSON schema and size limits; request ID; timeouts/rate limits; retry only idempotent requests

Identity provider → API

Relationship and purpose: Establish caller identity and role

Mechanism/protocol: OIDC/OAuth2, signed JWT validation or trusted identity proxy

Validation and connection handling: Verify issuer, audience, signature, expiry; reject missing/malformed credentials; cache keys safely; no client-provided authorization claims without verification

API → LangGraph orchestrator

Relationship and purpose: Start/resume graph with verified request context

Mechanism/protocol: In-process typed Python call if co-located; otherwise private HTTPS/JSON or gRPC

Validation and connection handling: Typed state contract; server-generated identity and thread scope; deadline propagation; authenticated service-to-service calls if separated

LangGraph → Intent router / subgraphs

Relationship and purpose: Select allowed route and execute workflow

Mechanism/protocol: In-process graph state transitions

Validation and connection handling: Structured intent schema; bounded transitions; refusal for unsupported tasks; authorization checked in tools even if routing was wrong

Orchestrator → Gemini on Vertex AI

Relationship and purpose: Planning, SQL generation, synthesis

Mechanism/protocol: Google authenticated HTTPS API via model SDK

Validation and connection handling: Structured output schemas, prompt version, allowed model ID; IAM/ADC service identity, deadlines, token budgets, bounded backoff on transient failures

Analysis → Schema/semantic catalog

Relationship and purpose: Discover approved tables, join paths and metric definitions

Mechanism/protocol: In-process catalog lookup; BigQuery metadata API/SQL or governed metadata store

Validation and connection handling: Allowlisted dataset/columns; cache with schema version; fail closed on unknown columns/metrics; read-only metadata privileges

Analysis → Golden Retriever → Vector index

Relationship and purpose: Retrieve analogous analyst examples

Mechanism/protocol: In-process call followed by authenticated Vertex AI Vector Search HTTPS/gRPC endpoint where supported

Validation and connection handling: Apply tenant/product/document ACL filters and review status before model context; top-k limits, relevance thresholds, timeouts; do not obey retrieved instructions

Golden ingestion Job → GCS

Relationship and purpose: Read authoritative Trio documents

Mechanism/protocol: Authenticated Cloud Storage JSON/gRPC client API over TLS

Validation and connection handling: IAM per bucket/prefix, object checksum/generation preconditions, schema validation, classification, PII review, idempotent ingestion

Golden ingestion Job → Embedding/Vector Search

Relationship and purpose: Embed and publish approved index entries

Mechanism/protocol: Authenticated Vertex AI APIs over TLS

Validation and connection handling: Fixed embedding model/version, metadata validation, duplicate detection, approval gate, index update auditing and retry/backoff

SQL generator → Validator

Relationship and purpose: Submit untrusted SQL for policy approval

Mechanism/protocol: Typed in-process function call

Validation and connection handling: GoogleSQL AST parsing, allowlisted statements/tables/columns, entitlement enforcement, aggregate privacy, query budget and dry-run validation

Validator/Runner → BigQuery

Relationship and purpose: Submit allowed read-only analytical job and retrieve bounded results

Mechanism/protocol: Google BigQuery API over HTTPS, typically via official Python client

Validation and connection handling: IAM least privilege, approved views, read-only query, maximum bytes billed, job timeout, parameter binding where applicable, row/result limits, job ID correlation

BigQuery → Analysis / synthesis

Relationship and purpose: Return query evidence or typed errors

Mechanism/protocol: Client-library response objects, sanitized in process

Validation and connection handling: Validate result schema/types, empty/null handling, PII/small-cohort controls, limits; never forward unrestricted raw transaction rows to LLM

LangGraph ↔ Cloud SQL checkpoints

Relationship and purpose: Persist/restore thread state and interrupts

Mechanism/protocol: PostgreSQL wire protocol over TLS through Cloud SQL connector/proxy

Validation and connection handling: Thread ownership and namespace, schema validation, transactional checkpoint writes, connection pooling with Cloud Run concurrency bounds, retention

Report management ↔ Cloud SQL reports

Relationship and purpose: Save, list, retrieve and soft-delete owned reports

Mechanism/protocol: PostgreSQL wire protocol over TLS

Validation and connection handling: Parameterized statements, row ownership predicates, transactions, optimistic concurrency/idempotency keys, access audit, bounded connection pools

Report management ↔ GCS report artifacts

Relationship and purpose: Store or retrieve large exports

Mechanism/protocol: Cloud Storage API over TLS

Validation and connection handling: User authorization before signed URLs or download, object metadata/ACL, retention and deletion consistency, generation preconditions

Approval gate ↔ checkpoint/report store

Relationship and purpose: Pause, freeze targets, resume and execute confirmed deletion

Mechanism/protocol: LangGraph interrupt/resume plus transactional PostgreSQL operations

Validation and connection handling: Exact IDs and operation token, expiry, ownership recheck, idempotent execution, audit, cancellation without side effects

Graph/tools → tracing and monitoring

Relationship and purpose: Record performance, failures and audit events

Mechanism/protocol: OpenTelemetry spans/exporters, HTTPS ingestion, Cloud Logging client

Validation and connection handling: Correlation IDs, sensitive-field redaction, bounded queues/retries, access and retention controls; telemetry failures should not leak data or crash chat

Cloud Run → Secret Manager/IAM

Relationship and purpose: Load secrets and acquire service credentials

Mechanism/protocol: Google-authenticated HTTPS APIs and runtime identity metadata

Validation and connection handling: Per-service least-privilege account, secret version/rotation, no .env secrets committed, no secret values in logs; cache minimally

Runtime → versioned persona config

Relationship and purpose: Load approved presentation style

Mechanism/protocol: Cloud SQL query or trusted configuration API

Validation and connection handling: Authorize edit/publish separately, schema validation, published version pinning and rollback, never permit persona to supersede security policy

Operational connection guidance. Cloud Run services are stateless and may scale concurrently; use pooled database connections with explicit limits to avoid exhausting Cloud SQL. Apply deadlines and cancellation to BigQuery/model calls. For shared external APIs, reuse managed client

instances when safe, cap in-flight calls and back off on throttling. Service-to-service authentication should use IAM identities rather than long-lived static passwords wherever possible. Validate data again at every trust boundary even when an upstream node already checked it.

3.3 Critical cross-requirement dependencies

R01 ↔ R04: SQL generation depends on authorization and PII policy

SQL generation may understand the user's question, but only the trusted validator and BigQuery access layer determine what may execute. Product entitlements shape the permitted query domain, and result privacy checks constrain what synthesis can see. **Purpose:** prevent unauthorized analytics and PII disclosure even if Gemini generates malicious or mistaken SQL. **Mechanism:** in-process typed SQL handoff, deterministic policy validation, authenticated BigQuery API, and post-query sanitization. **Validation:** AST policy, secure views/IAM, row/column allowlists, cohort thresholds and output review.

R01 ↔ R08: Golden examples improve planning but do not authorize queries

Relevant Golden Trios inform choice of metrics, decomposition patterns and SQL techniques. **Purpose:** reuse expert analytical practices. **Mechanism:** vector-retrieval API returning vetted metadata/context to the planning node. **Validation:** ACL and review-status filtering, relevance/reranking, schema-version checks, prompt-injection isolation. All generated SQL still passes R04 validation.

R02 ↔ R05: Durable conversation state enables safe confirmation

An approval request cannot rely only on a transient CLI variable. **Purpose:** survive pause/resume and retain the exact operation context without accidental deletion. **Mechanism:** LangGraph checkpointer, PostgreSQL pending-action record and trusted resume command. **Validation:** same authenticated user/thread, action ID, frozen target IDs, expiry and transactional recheck; approval is not inferred from free-form history alone.

R03 ↔ R05: Generated reports become managed, deletable artifacts

Report synthesis produces content while report management owns persistence and lifecycle. **Purpose:** ensure reports can be saved, retrieved, and deleted with ownership and audit. **Mechanism:** typed report object to Cloud SQL storage; optional GCS artifact reference. **Validation:** schema, owner, report ID, content PII scan and operation-specific authorization. Destructive operations are routed through R05, not handled as ordinary report text generation.

R06 ↔ R07: Recovery needs diagnostic traces and metrics

Typed error classifications and retry counters are recorded in traces. **Purpose:** explain why a run recovered, degraded or failed and identify expensive loops. **Mechanism:** in-process error events and OpenTelemetry/Cloud Logging exporters. **Validation:** consistent error taxonomy, correlation IDs, redaction, latency/cost tracking, bounded execution policy.

R09 ↔ R12: Personal preference and global persona are separate policy layers

The CEO's published tone and an individual's table/bullet preference both influence final presentation. **Purpose:** consistent corporate voice with personal usability. **Mechanism:** versioned configuration and user profile loaded from Cloud SQL into the synthesizer. **Validation:** priority rules and schema; neither may modify security settings, SQL permissions or metric definitions.

R10 ↔ R11 ↔ R08: Learning must be governed by evaluation

Observed failures and human corrections generate candidate improvements; evaluation decides whether they are safe and effective; approved examples may update the Golden index. **Purpose:** improve quality without self-reinforcing bad outputs. **Mechanism:** offline jobs, governed review, versioned artifact/index publication, CI/evaluation pipeline. **Validation:** benchmark thresholds, provenance, approval status, PII checks, regression and rollback.

R13 ↔ R04/R05/R07: Every extension inherits existing controls

Charts must use approved aggregates; email must verify recipients and report access; external trends must be sourced and treated as untrusted. **Purpose:** avoid reintroducing security or audit gaps through new tools. **Mechanism:** typed tool contracts and restricted service APIs. **Validation:** identity, content policy, action-specific consent, time/cost budgets, telemetry.

3.4 Runtime sequence and error branches

Question and identity: CLI/API obtains the question and verified user/product scope.

Classification and context: LangGraph restores the thread, chooses analysis/schema/report branch, and resolves follow-up references.

Knowledge: retrieve approved schema, metric definitions and, in production, relevant Golden Trios.

Plan and SQL: construct one or more analytical steps and produce candidate SQL.

Pre-execution gate: parse/authorize SQL; if unsafe, refuse; if correctable syntax/schema mismatch, invoke bounded repair and revalidate.

BigQuery execution: run with hard limits, record job metadata; retry only transient service errors.

Result assessment: accept legitimate empty results, or attempt justified bounded correction where the question/data mismatch is diagnosable.

Synthesis: convert sanctioned evidence to explanatory findings, hypotheses, caveats and actions; run output safety checks.

Persistence and response: save checkpoints, optionally save a report, return response through the API/CLI and record traces.

If deletion is requested: resolve only owned report IDs, show the target list, suspend via approval gate, and execute only after a scoped confirmation and revalidation.

Failure policy: Never retry authorization failures by weakening policy. Do not treat an empty result as proof that the SQL was wrong. Terminate gracefully when request budgets are reached and preserve a clear explanation of what succeeded or was unavailable.

---

4. Architectural decisions and assumptions register

The items below are explicit design proposals or unresolved questions, not assertions supplied by OpsFleet.

D01

Decision / assumption: Use LangGraph with deterministic security tools

Rationale and impact: Durable state, explicit branching, bounded retries and human interrupts

Status / follow-up: Proposed

D02

Decision / assumption: Start with one orchestrator and subgraphs, not independently autonomous agents

Rationale and impact: Simpler security, tracing, deployment and testing

Status / follow-up: Proposed; split only with evidence

D03

Decision / assumption: Python uv project and existing BigQuery Runner/Gemini adapter are retained

Rationale and impact: Build on tested baseline, reduce unnecessary rewrite

Status / follow-up: Verify actual code/interfaces

D04

Decision / assumption: CLI is the required POC interface; web UI is production optional

Rationale and impact: Matches assignment scope

Status / follow-up: Confirm final demo expectations

D05

Decision / assumption: Cloud Run compute is stateless; checkpoints/reports use PostgreSQL

Rationale and impact: Enables scale-out and reliable resume

Status / follow-up: Production design

D06

Decision / assumption: BigQuery is read-only, with controlled views/columns and execution budgets

Rationale and impact: Limits data access and financial risk

Status / follow-up: Define specific IAM/views

D07

Decision / assumption: Product entitlements live in a trusted application mapping

Rationale and impact: Public dataset lacks executive-to-product ownership

Status / follow-up: Client clarification needed

D08

Decision / assumption: PII is excluded/suppressed before model calls and final output

Rationale and impact: Protect raw customer data and reduce model exposure

Status / follow-up: Define PII classes/cohort threshold

D09

Decision / assumption: Revenue uses actual item sale values subject to status policy

Rationale and impact: Avoid treating catalog list price as realized sales

Status / follow-up: Metric policy needed

D10

Decision / assumption: Churn requires a defined activity/inactivity window

Rationale and impact: Not a direct theLook field; definitions change results

Status / follow-up: Client definition needed

D11

Decision / assumption: All report deletions use a frozen target set, explicit approval and soft delete

Rationale and impact: Recoverability and auditable safety

Status / follow-up: Confirm retention/permanent purge rules

D12

Decision / assumption: Golden Trios in GCS, approved index in Vertex AI retrieval

Rationale and impact: Separates durable sources from search index

Status / follow-up: Synthetic examples if no real corpus

D13

Decision / assumption: Offline, evaluated promotion of new knowledge/prompt changes

Rationale and impact: Avoid self-reinforcing hallucinations

Status / follow-up: Define review owners/thresholds

D14

Decision / assumption: Persona configuration is editable only for presentation

Rationale and impact: Prevent tone changes from overriding security

Status / follow-up: Define admin RBAC and precedence

D15

Decision / assumption: Use model abstraction and configurable Gemini model/timeouts

Rationale and impact: Model availability and throttling vary; 3.8 Flash timeout observed

Status / follow-up: Test quota/latency and fallback

D16

Decision / assumption: Bound SQL repairs, model calls, total tokens, BigQuery bytes and time

Rationale and impact: Prevent runaway loops and costs

Status / follow-up: Tune with measurements; e.g. 2 repairs initially

D17

Decision / assumption: Reports store evidence and provenance, not just prose

Rationale and impact: Supports auditing and correction

Status / follow-up: Define data retention and export rules

D18

Decision / assumption: Observability is privacy-safe by design

Rationale and impact: Traces could otherwise become uncontrolled PII copies

Status / follow-up: Define redaction and access policies

D19

Decision / assumption: GCP region, data residency, availability, backups and disaster recovery are not yet specified

Rationale and impact: Material production operational decisions

Status / follow-up: Client/platform input needed

D20

Decision / assumption: No exact POC completion status is asserted without repository/tests

Rationale and impact: Distinguishes design intention from demonstrated behavior

Status / follow-up: Audit and record test evidence

4.1 Items that should be explicitly clarified with OpsFleet

Which product-level mapping grants each executive access, and whether authorization must be by product ID, category, brand, department or tenant.

Whether user-level customer ranking is allowed at all given the strict PII rule; if allowed, which anonymous aggregates and minimum cohort sizes are acceptable.

Official definitions of net/gross revenue, returns, discounts, qualifying orders, active customers and churn.

Whether synthetic Golden Trios satisfy the design demonstration or whether the client can provide examples.

Whether report deletion is permanently destructive or recoverable, and whether exports must be deleted simultaneously.

Authentication expectations for a local CLI POC, and which GCP region/security policies apply to the proposed production architecture.

Intended volume/concurrency, latency objectives, query cost budget, report retention and service availability targets.

---

5. Validation strategy and proposed demonstration evidence

The submission should include runnable instructions, example chat transcripts, sanitized logs/traces and screenshots or terminal captures showing expected outcomes. The following tests are recommended minimum acceptance evidence; they are **not marked passed here**.

T01: Monthly revenue

Requirement(s): R01

Expected demonstrable outcome: Validated BigQuery query and independently checked revenue totals

T02: State/product comparison

Requirement(s): R01

Expected demonstrable outcome: Multi-query decomposition and evidence-linked answer

T03: Conversational follow-up

Requirement(s): R02

Expected demonstrable outcome: Correct inherited period/metric with new dimension/filter

T04: Quarterly report

Requirement(s): R03

Expected demonstrable outcome: Structured insights/actions, saved and retrieved by owner

T05: Direct PII request

Requirement(s): R04

Expected demonstrable outcome: No emails, addresses or other prohibited identifiers disclosed

T06: Cross-user product request

Requirement(s): R04

Expected demonstrable outcome: Unauthorized product scope blocked in data-access layer

T07: Delete, then cancel

Requirement(s): R05

Expected demonstrable outcome: Frozen list shown; reports remain intact

T08: Delete, then confirm

Requirement(s): R05

Expected demonstrable outcome: Only owned reports affected; audit emitted

T09: Invalid SQL injection

Requirement(s): R06

Expected demonstrable outcome: One or more bounded repairs; validation always reapplied

T10: Empty result

Requirement(s): R06

Expected demonstrable outcome: Correct absence explained or justified clarification; no fabricated data

T11: Gemini timeout/rate limit

Requirement(s): R06

Expected demonstrable outcome: Bounded retry/fallback or friendly recoverable failure

T12: Inspect trace

Requirement(s): R07

Expected demonstrable outcome: Correlated spans for classifier, model, SQL, error and response; no PII

For the production-only requirements, include design-review evidence: retrieval quality tests for Golden Trios, a sample preference profile, a versioned persona update/rollback example, a

model/regression evaluation rubric, and example extension interfaces. These need not all run in the POC.

---

6. Conclusion

6.1 System summary

OpsFleet's proposed assistant is a guarded, conversational analytical system: executives ask retail questions naturally; LangGraph plans and routes work; Gemini helps generate SQL and explain results; controlled BigQuery access supplies evidence; reports are saved under user ownership; destructive report actions require durable confirmation; and observable, bounded recovery protects the user experience. In production, Golden Knowledge, preferences, evaluations and persona versioning improve relevance and adaptability without weakening security.

6.2 Known limitations

The public theLook dataset is demonstrative rather than a governed production retail estate; it lacks native executive-product entitlements and may not support every requested causal explanation or churn definition.

SQL execution success is not equivalent to semantic correctness; report findings need independent numerical and grounding checks.

Privacy risk persists in small, linkable aggregates even when direct names/emails are removed; cohort and differencing controls require policy design.

The historical Golden Bucket is theoretical; retrieval quality cannot be evaluated realistically without curated examples.

Gemini latency, quota and pricing behavior can change; the observed timeout requires investigation and an explicit runtime budget.

Cloud Run, BigQuery, Vertex AI and PostgreSQL introduce network outages, quotas, cost and eventual consistency considerations; a production SLO and runbook are still needed.

Several diagrams simplify error cycles, authorization enforcement, background ingestion and output-safety checks. Detailed implementation diagrams should expose those paths.

Current POC code has not been examined here; required POC demonstration status cannot yet be certified.

6.3 Prioritized improvements and extensions

First: prove the mandatory POC vertical slice: SQL analysis, conversation, evidence-based reports, PII/product isolation, explicit deletion approval, bounded repair and traceability.

Next: formalize a semantic layer, automated reference-query tests and privacy-aware report assertions to reduce silently incorrect answers.

Then: introduce reviewed Golden Trios with ACL-aware hybrid retrieval, reranking and provenance.

Then: add governed user preference and persona stores, offline regression evaluation and controlled improvement promotion.

Finally: integrate visualization, PDF/export, email with recipient/action approval, scheduled reporting, and external research with attribution; harden cloud networking, backups, SLOs and audit retention.

The key architectural principle is that **generative flexibility lives inside deterministic boundaries**: the LLM can propose investigations and explanations, but trusted services decide what may be read, returned, stored, transmitted and deleted. This separation is what makes the prototype extensible toward a credible production system.